EE427 COMPUTER NETWORKS

Mini Project

CEILING FAN SPEED CONTROL

SUBMITTED By:-

Asmit Deshmukh – 231EE108

Bhamidipati Surya Venkata Vivek – 231EE109

Chittela Naga Sai Sree Sailesh – 231EE110

Devashish Singh – 231EE111

SUBMITTED TO –

Dr. K.P. Vittal

(Course Instructor)

INTRODUCTION

The AI-Based IoT Ceiling Fan Speed Control System is a smart home project that lets users control a ceiling fan wirelessly through a web interface and AI commands. The system uses the ESP8266 microcontroller, programmed with the Arduino IDE. It functions as a Wi-Fi Access Point, allowing devices to connect directly without an external Wi-Fi network.

The system offers four speed modes:
- OFF (0%)
- LOW (10%)
- MEDIUM (55%)
- HIGH (95%)

It controls the fan speed with Pulse Width Modulation (PWM) for smooth and efficient operation.

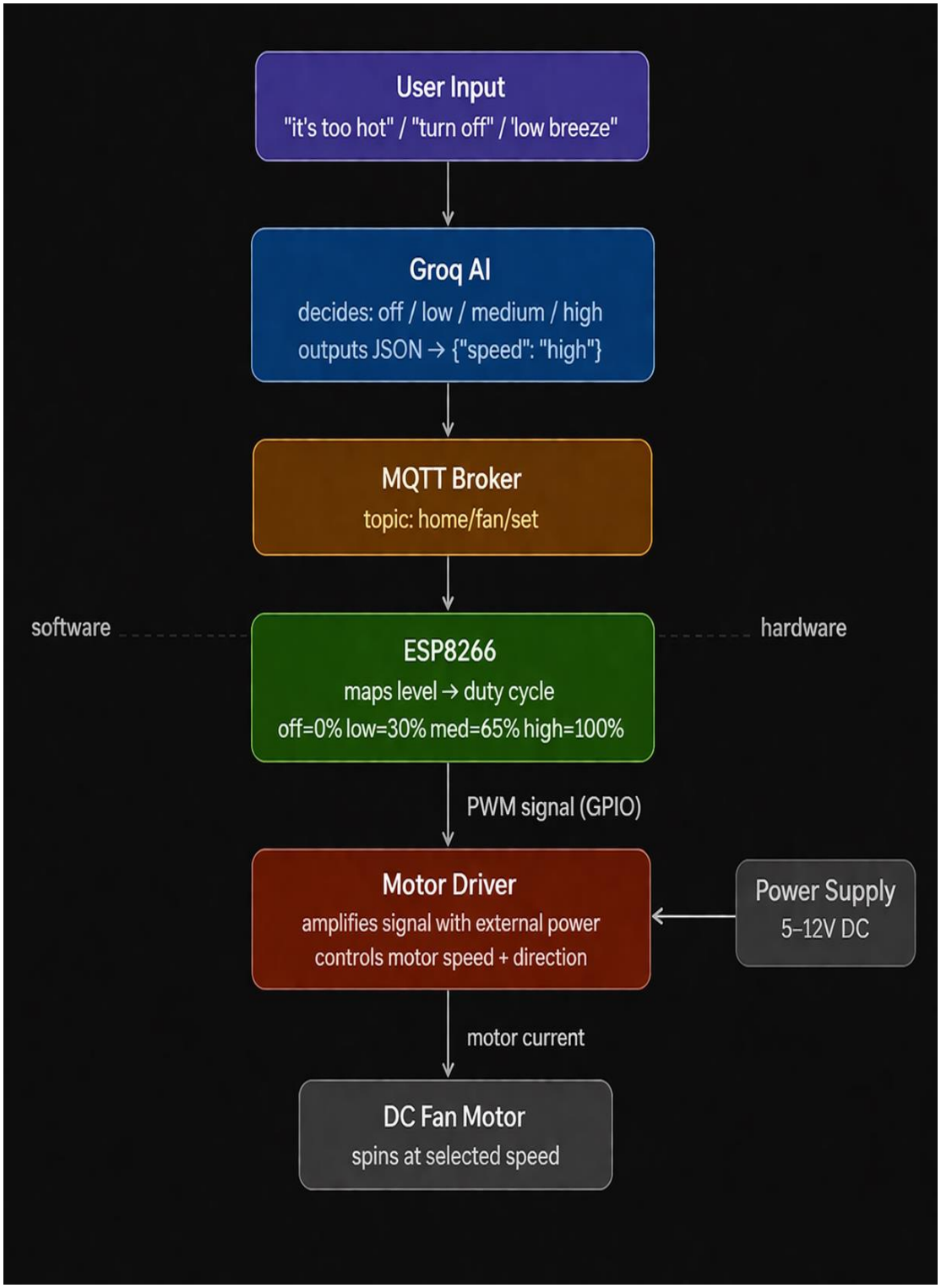
AI Integration Using Groq API
The project includes AI features through the Groq API, enabling users to give natural language commands, such as:
- "Turn off the fan"
- "Set fan to medium"
- "Set fan to high speed"
- "Turn fan to low"

The AI interprets the user's command and translates it into one of the four speed settings. The chosen speed is then sent to the ESP8266, which adjusts the fan speed using PWM.

Web Interface Features
- Simple, user-friendly interface
- Four speed control buttons
- Real-time fan speed updates
- AI command input support
- Wireless control via ESP8266 Access Point

This project shows how Artificial Intelligence and IoT can come together in smart home automation. By integrating ESP8266, Arduino IDE, Groq API, and PWM control, the system offers a smart and modern solution for managing ceiling fan speed. It replaces traditional regulators with AI-driven controls, making home appliances more convenient and flexible.

Flow Diagram (Flow of Execution)



CODING ESP8266

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

// ── WiFi credentials ──────────────────────────────────

const char* WIFI_SSID     = "********;
const char* WIFI_PASSWORD = "********";

// ── MQTT broker settings ──────────────────────────────
const char* MQTT_BROKER   = "broker.hivemq.com";
const int   MQTT_PORT     = 1883;
const char* MQTT_TOPIC    = "home/fan/set";
const char* MQTT_STATUS   = "home/fan/status";
const char* MQTT_CLIENT_ID = "esp8266_fan_001";

// ── ThingSpeak settings ───────────────────────────────
const char* TS_API_KEY    = "04O2PW7EKU2BZSWF;
const char* TS_URL        = "http://api.thingspeak.com/update";
const unsigned long TS_INTERVAL = 15000;

// ── Pin definitions ───────────────────────────────────
const int PIN_ENA  = D1;
const int PIN_IN1  = D2;
const int PIN_IN2  = D3;
// const int PIN_DHT  = D4;

// ── PWM duty cycle values for each speed level ────────
//    ESP8266 analogWrite range: 0–1023
const int PWM_OFF    = 0;    // 0%   — fan stopped
const int PWM_LOW    = 307;  // 30%  — gentle breeze
const int PWM_MEDIUM = 665;  // 65%  — moderate
const int PWM_HIGH   = 1023; // 100% — full speed

// ── DHT22 sensor (uncomment if using) ─────────────────
// #define DHTTYPE DHT22
// DHT dht(PIN_DHT, DHTTYPE);

// ── Global state ──────────────────────────────────────
WiFiClient   wifiClient;
PubSubClient mqttClient(wifiClient);

String  currentSpeedLevel = "off";  // tracks current fan level
int     currentPWM        = 0;      // tracks current PWM value
```

```
unsigned long lastThingSpeakTime = 0;

// ────────────────────────────────────────────────────────────
// Connect to WiFi
// ────────────────────────────────────────────────────────────
void connectWiFi() {
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    attempts++;
    if (attempts > 40) {
      Serial.println("\nWiFi failed! Restarting...");
      ESP.restart();
    }
  }

  Serial.println("\nWiFi connected!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

// ────────────────────────────────────────────────────────────
// Set fan speed based on level string
// ────────────────────────────────────────────────────────────
void setFanSpeed(String level) {
  level.toLowerCase();
  level.trim();

  int pwmValue = 0;

  if (level == "off") {
    pwmValue = PWM_OFF;
  } else if (level == "low") {
    pwmValue = PWM_LOW;
  } else if (level == "medium") {
    pwmValue = PWM_MEDIUM;
  } else if (level == "high") {
    pwmValue = PWM_HIGH;
  } else {
    Serial.print("Unknown speed level received: ");
    Serial.println(level);
    return;
  }

  analogWrite(PIN_ENA, pwmValue);
```

```
  currentSpeedLevel = level;
  currentPWM        = pwmValue;

  Serial.print("Fan set to: ");
  Serial.print(level);
  Serial.print("  (PWM=");
  Serial.print(pwmValue);
  Serial.println(")");

  String statusMsg = "{\"speed\":\"" + level + "\",\"pwm\":" + String(pwmValue) + "}";
  mqttClient.publish(MQTT_STATUS, statusMsg.c_str(), true);
}

// ──────────────────────────────────────────────────────────
// MQTT message received callback
// ──────────────────────────────────────────────────────────
void onMqttMessage(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("MQTT received on [");
  Serial.print(topic);
  Serial.print("]: ");
  Serial.println(message);

  // Parse JSON: expected format {"speed": "high"}
  StaticJsonDocument<128> doc;
  DeserializationError error = deserializeJson(doc, message);

  if (error) {
    Serial.print("JSON parse error: ");
    Serial.println(error.c_str());
    return;
  }

  if (doc.containsKey("speed")) {
    String speed = doc["speed"].as<String>();
    setFanSpeed(speed);
  } else {
    Serial.println("JSON missing 'speed' key");
  }
}

// ──────────────────────────────────────────────────────────
// Connect to MQTT broker
// ──────────────────────────────────────────────────────────
void connectMQTT() {
  while (!mqttClient.connected()) {
    Serial.print("Connecting to MQTT broker...");
```

```
    if (mqttClient.connect(MQTT_CLIENT_ID)) {
      Serial.println("connected!");
      mqttClient.subscribe(MQTT_TOPIC);
      Serial.print("Subscribed to: ");
      Serial.println(MQTT_TOPIC);

      // Announce online
      mqttClient.publish(MQTT_STATUS, "{\"status\":\"online\",\"speed\":\"off\"}",
true);
    } else {
      Serial.print("failed, rc=");
      Serial.print(mqttClient.state());
      Serial.println(" — retrying in 3 seconds");
      delay(3000);
    }
  }
}

// ──────────────────────────────────────────────────────────────
// Send data to ThingSpeak
// ──────────────────────────────────────────────────────────────
void sendToThingSpeak() {
  if (WiFi.status() != WL_CONNECTED) return;

  int speedNum = 0;
  if      (currentSpeedLevel == "low")    speedNum = 1;
  else if (currentSpeedLevel == "medium") speedNum = 2;
  else if (currentSpeedLevel == "high")   speedNum = 3;


  String url = String(TS_URL) +
               "?api_key=" + TS_API_KEY +
               "&field1="  + String(speedNum);

  WiFiClient client;
  HTTPClient http;
  http.begin(client, url);

  int httpCode = http.GET();

  if (httpCode == 200) {
    Serial.println("ThingSpeak updated OK");
  } else {
    Serial.print("ThingSpeak error: ");
    Serial.println(httpCode);
  }

  http.end();
}

// ──────────────────────────────────────────────────────────────
```

```
// SETUP
// ──────────────────────────────────────────────────────────
void setup() {
  Serial.begin(115200);
  Serial.println("\n\n=== DC Fan Controller — ESP8266 ===");

  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);


  pinMode(PIN_ENA, OUTPUT);
  analogWrite(PIN_ENA, PWM_OFF);
  analogWriteFreq(1000);
  analogWriteRange(1023);



  connectWiFi();

  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(onMqttMessage);

  connectMQTT();

  Serial.println("=== Setup complete. Waiting for commands... ===\n");
}

// ──────────────────────────────────────────────────────────
// LOOP
// ──────────────────────────────────────────────────────────
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi lost — reconnecting...");
    connectWiFi();
  }

  if (!mqttClient.connected()) {
    connectMQTT();
  }

  mqttClient.loop();
  unsigned long now = millis();
  if (now - lastThingSpeakTime >= TS_INTERVAL) {
    lastThingSpeakTime = now;
    sendToThingSpeak();
  }
}
```

CODING FRONTEND AND BACKEND

```
import json
import threading
import time
from flask import Flask, render_template_string
from flask_socketio import SocketIO, emit
import paho.mqtt.client as mqtt
from groq import Groq

# ── Configuration ──────────────────────────────────────────
GROQ_API_KEY  = "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"
MQTT_BROKER   = "broker.hivemq.com"
MQTT_PORT     = 1883
MQTT_TOPIC    = "home/fan/set"
MQTT_STATUS   = "home/fan/status"

# ── Groq AI system prompt ──────────────────────────────────
SYSTEM_PROMPT = """
You are a fan speed controller AI.
The user will describe how they feel or what they want.
You must respond with ONLY one of these four words — nothing else:
  off
  low
  medium
  high

Rules:
- "off", "stop", "turn off", "I'm cold", "too cold" → off
- "a little", "slight breeze", "low", "gentle" → low
- "medium", "normal", "moderate", "okay speed" → medium
- "hot", "very hot", "max", "high", "full speed", "boiling" → high
- If unsure, pick medium
- NEVER explain yourself. Output only one word.
"""

# ── Flask + SocketIO app ───────────────────────────────────
app = Flask(__name__)
app.config['SECRET_KEY'] = 'fansecret123'
socketio = SocketIO(app, cors_allowed_origins="*")

# ── State ──────────────────────────────────────────────────
current_speed  = "off"
groq_client    = Groq(api_key=GROQ_API_KEY)
mqtt_client    = mqtt.Client()

# ── HTML template (web UI) ────────────────────────────────
```

```
HTML = """
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AI Fan Controller</title>
  <script src="https://cdn.socket.io/4.6.1/socket.io.min.js"></script>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: system-ui, sans-serif;
      background: #0f0f0f;
      color: #f0f0f0;
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
    }

    .card {
      background: #1a1a1a;
      border: 1px solid #2a2a2a;
      border-radius: 16px;
      padding: 40px;
      width: 100%;
      max-width: 500px;
    }

    h1 {
      font-size: 22px;
      font-weight: 600;
      margin-bottom: 6px;
      color: #ffffff;
    }

    .subtitle {
      font-size: 13px;
      color: #666;
      margin-bottom: 32px;
    }

    /* Fan speed display */
    .speed-display {
      display: flex;
      gap: 10px;
      margin-bottom: 28px;
    }

    .speed-pill {
      flex: 1;
```

```
      padding: 10px 0;
      border-radius: 8px;
      text-align: center;
      font-size: 13px;
      font-weight: 500;
      background: #252525;
      color: #555;
      border: 1px solid #2a2a2a;
      transition: all 0.3s;
    }

    .speed-pill.active-off    { background:#2a2a2a; color:#fff; border-color:#444; }
    .speed-pill.active-low    { background:#0d3b2e; color:#4ade80; border-
color:#166534; }
    .speed-pill.active-medium { background:#2d2000; color:#fbbf24; border-
color:#854d0e; }
    .speed-pill.active-high   { background:#3b0a0a; color:#f87171; border-
color:#991b1b; }

    /* Fan icon */
    .fan-icon {
      font-size: 64px;
      text-align: center;
      margin-bottom: 20px;
      transition: transform 0.3s;
      user-select: none;
    }

    .fan-icon.spin-slow   { animation: spin 2s linear infinite; }
    .fan-icon.spin-medium { animation: spin 0.8s linear infinite; }
    .fan-icon.spin-fast   { animation: spin 0.3s linear infinite; }

    @keyframes spin { from { transform: rotate(0deg); } to { transform: rotate(360deg);
} }

    /* Input area */
    label {
      display: block;
      font-size: 13px;
      color: #888;
      margin-bottom: 8px;
    }

    .input-row {
      display: flex;
      gap: 10px;
      margin-bottom: 16px;
    }

    input[type="text"] {
      flex: 1;
      background: #252525;
```

```
  border: 1px solid #333;
  border-radius: 8px;
  padding: 12px 16px;
  font-size: 15px;
  color: #f0f0f0;
  outline: none;
  transition: border-color 0.2s;
}

input[type="text"]:focus { border-color: #555; }

button {
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 12px 20px;
  font-size: 15px;
  font-weight: 500;
  cursor: pointer;
  transition: background 0.2s;
  white-space: nowrap;
}

button:hover    { background: #1d4ed8; }
button:disabled { background: #333; color: #666; cursor: not-allowed; }

/* Quick buttons */
.quick-btns {
  display: flex;
  gap: 8px;
  margin-bottom: 24px;
  flex-wrap: wrap;
}

.quick-btn {
  flex: 1;
  min-width: 80px;
  background: #252525;
  color: #aaa;
  border: 1px solid #333;
  border-radius: 8px;
  padding: 8px 0;
  font-size: 13px;
  cursor: pointer;
  transition: all 0.2s;
}

.quick-btn:hover { background: #333; color: #fff; border-color: #555; }

/* Log */
.log-label {
```

```
    font-size: 12px;
    color: #555;
    margin-bottom: 8px;
    display: flex;
    justify-content: space-between;
  }

  .log {
    background: #111;
    border: 1px solid #222;
    border-radius: 8px;
    padding: 12px;
    font-size: 13px;
    font-family: monospace;
    height: 140px;
    overflow-y: auto;
    color: #666;
  }

  .log-entry { padding: 2px 0; border-bottom: 1px solid #1a1a1a; }
  .log-entry:last-child { border-bottom: none; }
  .log-entry .time { color: #444; margin-right: 8px; }
  .log-entry .msg  { color: #888; }
  .log-entry .speed-tag {
    font-weight: bold;
    padding: 1px 6px;
    border-radius: 4px;
    font-size: 11px;
  }
  .tag-off    { background:#2a2a2a; color:#aaa; }
  .tag-low    { background:#0d3b2e; color:#4ade80; }
  .tag-medium { background:#2d2000; color:#fbbf24; }
  .tag-high   { background:#3b0a0a; color:#f87171; }

  .thingspeak-link {
    display: block;
    text-align: center;
    margin-top: 20px;
    font-size: 12px;
    color: #444;
    text-decoration: none;
  }
  .thingspeak-link:hover { color: #888; }

  .status-dot {
    display: inline-block;
    width: 8px; height: 8px;
    border-radius: 50%;
    background: #666;
    margin-right: 6px;
    transition: background 0.3s;
  }
```

```
    .status-dot.connected { background: #4ade80; }
  </style>
</head>
<body>
<div class="card">

  <h1>🤖 AI Fan Controller</h1>
  <p class="subtitle">
    <span class="status-dot" id="dot"></span>
    <span id="conn-status">Connecting...</span>
  </p>

  <!-- Fan animation -->
  <div class="fan-icon" id="fan-icon">💨</div>

  <!-- Speed level pills -->
  <div class="speed-display">
    <div class="speed-pill" id="pill-off">Off</div>
    <div class="speed-pill" id="pill-low">Low</div>
    <div class="speed-pill" id="pill-medium">Medium</div>
    <div class="speed-pill" id="pill-high">High</div>
  </div>

  <!-- Text input -->
  <label>Tell the AI what you need:</label>
  <div class="input-row">
    <input type="text" id="user-input" placeholder="e.g. it's too hot, slight breeze,
turn off..." maxlength="200"/>
    <button id="send-btn" onclick="sendCommand()">Send</button>
  </div>

  <!-- Quick buttons -->
  <div class="quick-btns">
    <button class="quick-btn" onclick="quickSend('turn off the fan')">Off</button>
    <button class="quick-btn" onclick="quickSend('give me a gentle
breeze')">Low</button>
    <button class="quick-btn" onclick="quickSend('medium speed
please')">Medium</button>
    <button class="quick-btn" onclick="quickSend('it is very hot max
speed')">High</button>
  </div>

  <!-- Log -->
  <div class="log-label">
    <span>Activity log</span>
    <span id="esp-status" style="color:#555">ESP8266: unknown</span>
  </div>
  <div class="log" id="log"></div>

  <a class="thingspeak-link" href="https://thingspeak.com/channels/3322642"
target="_blank">
    📊 View ThingSpeak Dashboard →
```

```
  </a>

</div>

<script>
  const socket = io();
  let currentSpeed = 'off';

  socket.on('connect', () => {
    document.getElementById('dot').classList.add('connected');
    document.getElementById('conn-status').textContent = 'Connected';
  });

  socket.on('disconnect', () => {
    document.getElementById('dot').classList.remove('connected');
    document.getElementById('conn-status').textContent = 'Disconnected';
  });

  socket.on('speed_update', (data) => {
    updateUI(data.speed, data.user_input, data.ai_reasoning);
  });

  socket.on('esp_status', (data) => {
    document.getElementById('esp-status').textContent = 'ESP8266: ' + data.speed;
  });

  socket.on('error_msg', (data) => {
    addLog('⚠ ' + data.message, null);
    document.getElementById('send-btn').disabled = false;
    document.getElementById('send-btn').textContent = 'Send';
  });

  function updateUI(speed, userInput, reasoning) {
    currentSpeed = speed;

    // Update pills
    ['off','low','medium','high'].forEach(s => {
      const p = document.getElementById('pill-' + s);
      p.className = 'speed-pill' + (s === speed ? ' active-' + speed : '');
    });

    // Update fan icon animation
    const icon = document.getElementById('fan-icon');
    icon.className = 'fan-icon';
    if (speed === 'low')    icon.classList.add('spin-slow');
    if (speed === 'medium') icon.classList.add('spin-medium');
    if (speed === 'high')   icon.classList.add('spin-fast');

    // Add log entry
    if (userInput) addLog(userInput, speed);

    document.getElementById('send-btn').disabled = false;
```

```
    document.getElementById('send-btn').textContent = 'Send';
  }

  function addLog(text, speed) {
    const log  = document.getElementById('log');
    const now  = new Date().toLocaleTimeString();
    const tag  = speed ? `<span class="speed-tag tag-${speed}">${speed}</span>` : '';
    const div  = document.createElement('div');
    div.className = 'log-entry';
    div.innerHTML = `<span class="time">${now}</span><span class="msg">${text}</span>
${tag}`;
    log.prepend(div);
  }

  function sendCommand() {
    const input = document.getElementById('user-input').value.trim();
    if (!input) return;
    document.getElementById('send-btn').disabled = true;
    document.getElementById('send-btn').textContent = 'Thinking...';
    socket.emit('user_command', { text: input });
    document.getElementById('user-input').value = '';
  }

  function quickSend(text) {
    document.getElementById('user-input').value = text;
    sendCommand();
  }

  document.getElementById('user-input').addEventListener('keydown', (e) => {
    if (e.key === 'Enter') sendCommand();
  });
</script>
</body>
</html>
"""

# ────────────────────────────────────────────────────────────
# Groq AI — decide fan speed from user input
# ────────────────────────────────────────────────────────────
def ask_groq(user_input: str) -> str:
    """
    Send user input to Groq AI.
    Returns one of: 'off', 'low', 'medium', 'high'
    """
    try:
        response = groq_client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user",   "content": user_input}
            ],
            max_tokens=5,
```

```python
            temperature=0.1,
        )

        speed = response.choices[0].message.content.strip().lower()

        if speed not in ["off", "low", "medium", "high"]:
            print(f"[Groq] Unexpected response: '{speed}' — defaulting to medium")
            speed = "medium"

        print(f"[Groq] User: '{user_input}' → Speed: '{speed}'")
        return speed

    except Exception as e:
        print(f"[Groq] Error: {e}")
        return "off"

# ────────────────────────────────────────────────────────────
# MQTT — publish command to ESP8266
# ────────────────────────────────────────────────────────────
def publish_speed(speed: str):
    """Publish fan speed command as JSON to MQTT broker."""
    payload = json.dumps({"speed": speed})
    result  = mqtt_client.publish(MQTT_TOPIC, payload, qos=1, retain=True)

    if result.rc == mqtt.MQTT_ERR_SUCCESS:
        print(f"[MQTT] Published: {payload}")
    else:
        print(f"[MQTT] Publish failed: rc={result.rc}")

# ────────────────────────────────────────────────────────────
# MQTT callbacks
# ────────────────────────────────────────────────────────────
def on_mqtt_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[MQTT] Connected to broker")
        client.subscribe(MQTT_STATUS)
        print(f"[MQTT] Subscribed to {MQTT_STATUS}")
    else:
        print(f"[MQTT] Connection failed: rc={rc}")

def on_mqtt_message(client, userdata, msg):
    """Handle status messages coming back from ESP8266."""
    try:
        payload = json.loads(msg.payload.decode())
        speed   = payload.get("speed", "unknown")
        print(f"[MQTT] ESP8266 status: {payload}")
        socketio.emit('esp_status', {'speed': speed})
    except Exception as e:
        print(f"[MQTT] Status parse error: {e}")

def on_mqtt_disconnect(client, userdata, rc):
    print(f"[MQTT] Disconnected: rc={rc} — reconnecting...")
```

```python
# ────────────────────────────────────────────────────────────
# SocketIO — handle web UI events
# ────────────────────────────────────────────────────────────
@socketio.on('user_command')
def handle_command(data):
    user_input = data.get('text', '').strip()
    if not user_input:
        emit('error_msg', {'message': 'Empty input'})
        return

    print(f"\n[UI] User said: '{user_input}'")
    def process():
        speed = ask_groq(user_input)
        publish_speed(speed)
        socketio.emit('speed_update', {
            'speed':      speed,
            'user_input': user_input,
        })

    thread = threading.Thread(target=process, daemon=True)
    thread.start()

# ────────────────────────────────────────────────────────────
# Flask route
# ────────────────────────────────────────────────────────────
@app.route('/')
def index():
    return render_template_string(HTML)

# ────────────────────────────────────────────────────────────
# Main
# ────────────────────────────────────────────────────────────
def start_mqtt():
    mqtt_client.on_connect    = on_mqtt_connect
    mqtt_client.on_message    = on_mqtt_message
    mqtt_client.on_disconnect = on_mqtt_disconnect
    mqtt_client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
    mqtt_client.loop_forever()

if __name__ == '__main__':
    print("=" * 50)
    print("  AI Fan Controller — Backend")
    print("=" * 50)
    print(f"  MQTT broker : {MQTT_BROKER}:{MQTT_PORT}")
    print(f"  Web UI      : http://localhost:5000")
    print("=" * 50)
    mqtt_thread = threading.Thread(target=start_mqtt, daemon=True)
    mqtt_thread.start()
    time.sleep(1)
    socketio.run(app, host='0.0.0.0', port=5000, debug=False)
```

System Overview (Additional Information)

Communication Protocols

- MQTT Protocol
  - Publish–Subscribe architecture
  - Lightweight and efficient
  - Works over TCP/IP
- Wi-Fi (ESP8266)
  - Enables wireless communication

MQTT Features

- Topic-based messaging (home/fan/set)
- Low latency communication
- Reliable delivery (QoS levels)
- Decouples sender and receiver
- Suitable for IoT applications

Performance Characteristics

- Fast response time (near real-time)
- Low power consumption
- Efficient data transfer (MQTT)
- Scalable for multiple devices
- Stable operation with Wi-Fi connectivity

Advantages

- User-friendly (natural language control)
- Wireless and remote operation
- Low-cost implementation
- Modular and expandable system

Limitations

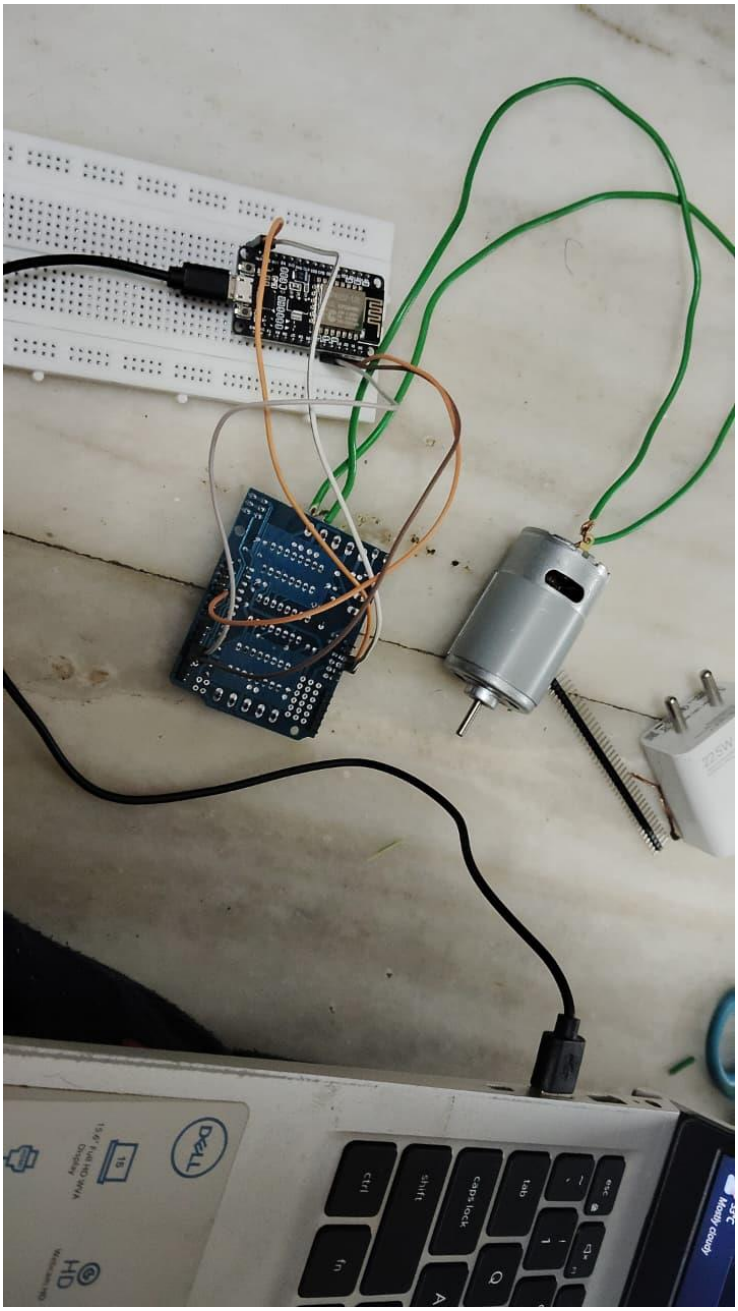
- Depends on internet connectivity
- AI response accuracy may vary
- MQTT broker reliability required

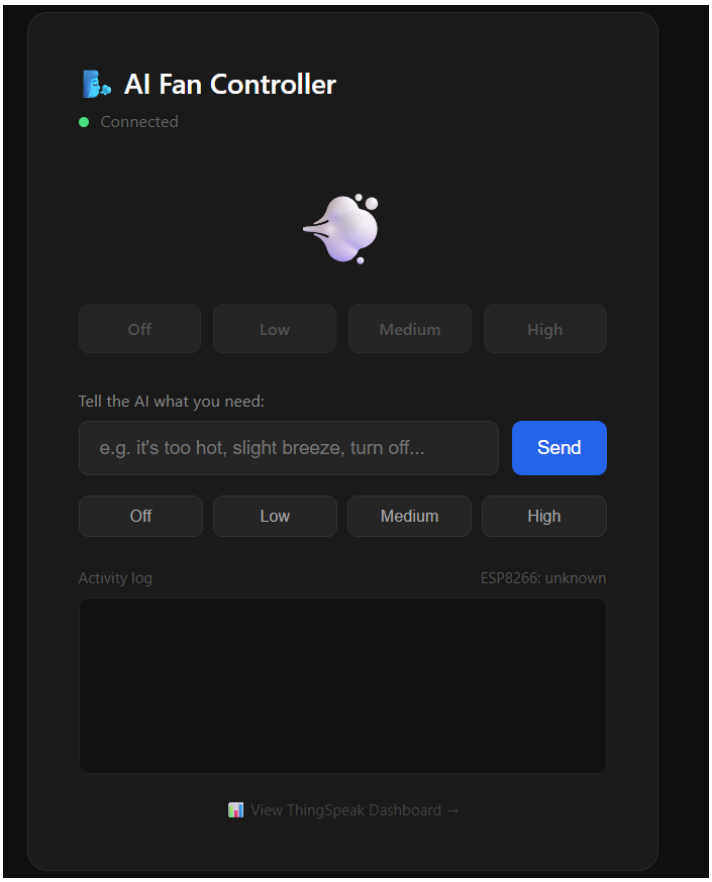
Future Enhancements

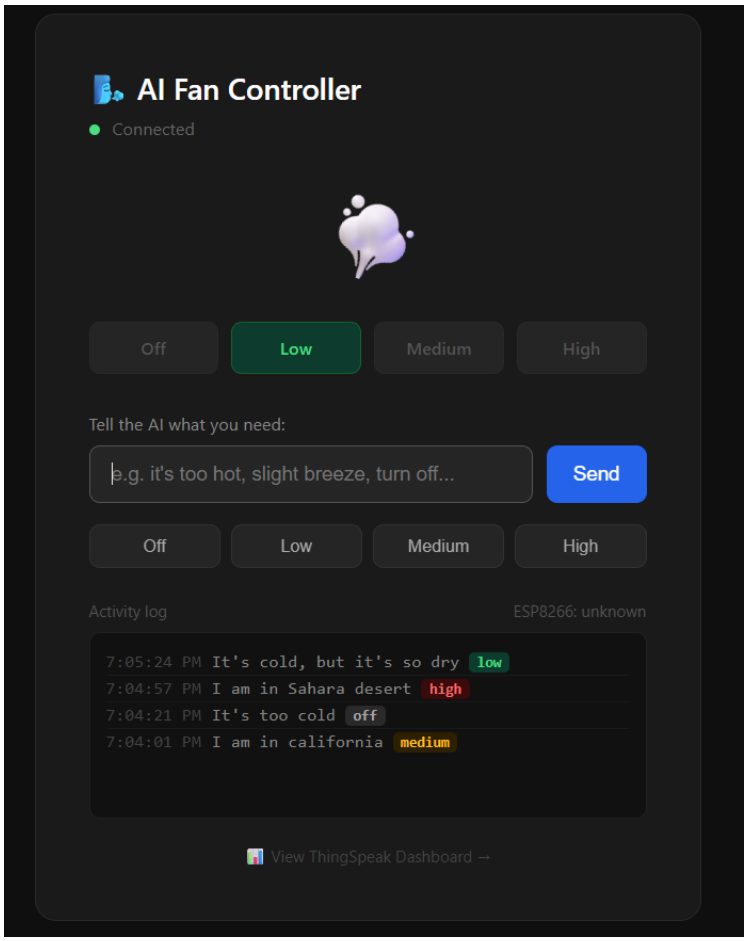
- Voice control integration, multi device smart home integration
- Mobile app interface, temperature based automatic control

RESULTS

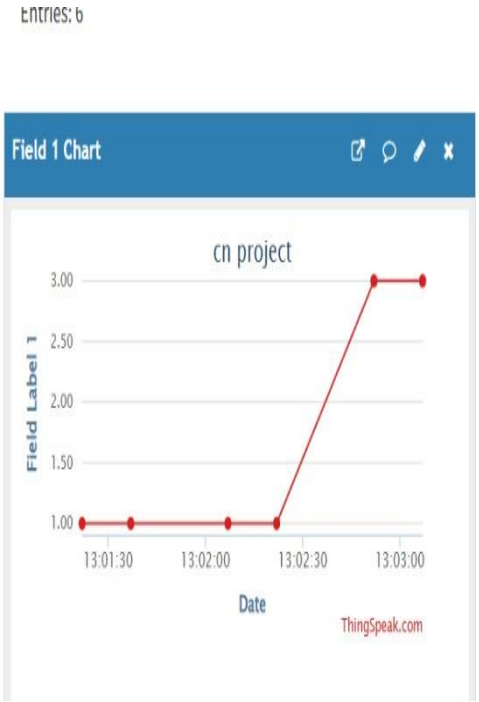
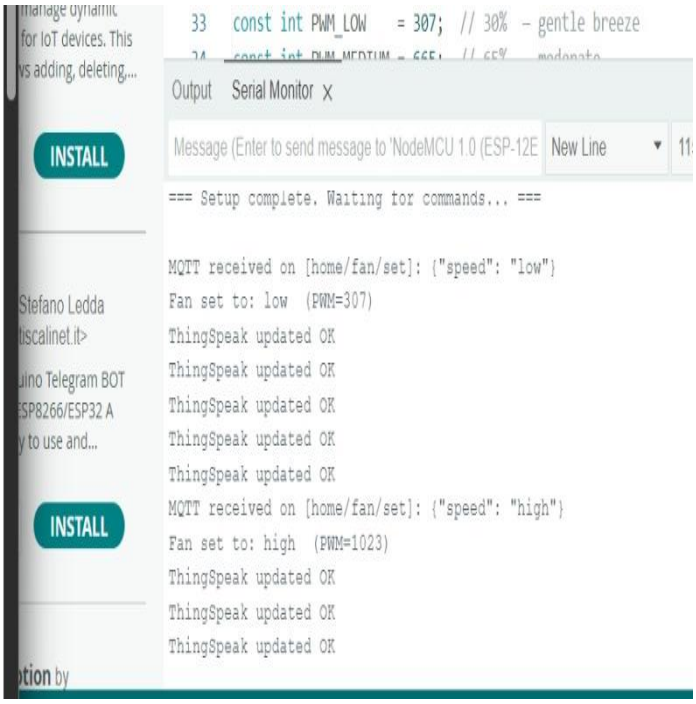
This project demonstrates an IoT-based AI fan controller using the ESP8266 for wireless communication and the L293D motor driver for efficient motor control. By integrating the Groq API, the system enables intelligent decision-making for automated fan operation based on user input or conditions. The design ensures low-cost, scalable, and real-time control suitable for smart home applications. Overall, it highlights the effective combination of IoT and AI to improve energy efficiency and user comfort.




AI Fan Controller
Connected
Off
Low
Medium
High
Tell the AI what you need:
e.g. it's too hot, slight breeze, turn off...
Send
Off
Low
Medium
High
Activity log
ESP8266: unknown
View ThingSpeak Dashboard →


AI Fan Controller
Connected
Off
Low
Medium
High
Tell the AI what you need:
e.g. it's too hot, slight breeze, turn off...
Send
Off
Low
Medium
High
Activity log
ESP8266: unknown
7:05:24 PM It's cold, but it's so dry low
7:04:57 PM I am in Sahara desert high
7:04:21 PM It's too cold off
7:04:01 PM I am in california medium
View ThingSpeak Dashboard →

```
WiFi connected!
IP address: 10.70.65.10
Connecting to MQTT broker...connected!
Subscribed to: home/fan/set
=== Setup complete. Waiting for commands... ===

MQTT received on [home/fan/set]: {"speed": "low"}
Fan set to: low  (PWM=307)
ThingSpeak updated OK
```

```
MQTT received on [home/fan/set]: {"speed": "high"}
Fan set to: high  (PWM=1023)
```